IMPLEMENTATION PLAN · GATED DELIVERY

SeekAndDestroy

Step by step from schema to production, with an exit checklist on every phase. No phase begins until the phase above it is fully ticked.

Revision: 2 September 2026 Owners: e7 (build) · 2d (retrieval & scoring) · f8 (analytics) Companion: PROJECT_GOAL.pdf

> **The rule that governs this document.** A phase is complete when every box in its exit checklist is ticked and demonstrable — not when the code exists. Partial completion is not progress; it is the state that produced a golden set that could not fail, an audit table nobody wrote to, and an observability stack that was never switched on.

Ownership split

AGENT	OWNS	DOES NOT TOUCH
e7	Schema and migrations, seed generation, retrieval pipeline and indexing, deployment, guardrails, RBAC, telemetry and cost, release gating	Retrieval golden set, scoring weights and rules, the analytics layer
2d	Retrieval evaluation and golden set, dense/sparse/hybrid measurement, resiliency and change-risk scoring on the CI graph, eligibility rules, the UI	Schema, seed, indexing pipeline, deployment
f8	Aggregate analytics: CMDB health, impact analysis, incident and change insight, the constrained query layer	Retrieval internals, schema, seed, scoring, deployment. Read-only against `sad.*`

One working tree, three agents. Stage by filename, never `git add -A`. Announce a file before editing it if another agent could plausibly be in it.

PHASE 0 Contracts and plan

E7 · BLOCKS EVERYTHING

Nothing is written until the shape is agreed, because the expensive mistakes in this project have all been shape mistakes discovered late — three nullable foreign keys, an unfalsifiable test case, a corpus with no prose in it.

EXIT CHECKLIST

- ☐ CI class list agreed and closed: which classes exist, which are new tables, which are existing tables gaining a `CiId`
- ☐ Relationship types enumerated with parent/child direction stated explicitly, not implied
- ☐ Derived-column policy written down: which denormalised columns survive, fed from what, documented as derived at the point of definition
- ☐ DDL circulated to 2d and f8 before any data exists
- ☐ Neighbourhood-to-data-centre ratio decided and recorded

PHASE 1 Schema — migration 008

E7 · BLOCKS 2, 3, F8

Work

- `sad.ConfigurationItem` — identity, class, lifecycle, ownership, location, classification, discovery fields
- Class tables: new (`CiVmInstance`, `CiDatabaseInstance`, `CiBusinessService`, `CiLoadBalancer`, `CiDataCentre`, `CiNeighbourhood`) and existing tables gaining `CiId`
- `sad.CiRelationshipType`, `sad.CiRelationship`, `sad.TaskCi`
- `CmdbCiId` and `BusinessServiceCiId` on Incident, Change, Problem
- Backfill: every existing application, cluster and node gets a CI row

EXIT CHECKLIST

- ☐ Migration is idempotent — running it twice is a no-op, verified by running it twice
- ☐ `SET QUOTED_IDENTIFIER ON` at the top; any filtered index requires it at creation *and* at every later insert
- ☐ Every foreign key, check constraint and index named explicitly, never auto-named
- ☐ Derived columns carry a comment at the point of definition saying what feeds them
- ☐ Rollback path documented, even if the rollback is "restore and re-seed"
- ☐ New grants added to `provision-databases.sh` in the same change — a grant that exists only in a manual step is a production outage waiting for the next reset
- ☐ DDL sent to 2d and f8

PHASE 2 Seed

E7 · UNBLOCKS F8

Work

- One CI per item; typed relationships; the VM layer between application and physical host
- Containment edges acyclic by construction; dependency edges permitted to cycle, with one deliberate cycle planted
- Deliberate CMDB defects: orphan CIs, stale CIs, duplicates, missing owners
- Every fixture exported through `SCENARIOS`

EXIT CHECKLIST

- ☐ Generation is byte-for-byte deterministic — generate twice, compare files
- ☐ No test anywhere asserts a literal count that the generator produces; all assertions read `SCENARIOS`
- ☐ Every planted defect class is present and non-zero
- ☐ The dependency cycle exists and is exported as a fixture
- ☐ Containment graph verified acyclic by query, not by assumption
- ☐ Derived columns agree with the graph — zero rows where a cluster's data centre disagrees with its neighbourhood's
- ☐ Seed loads into an empty database with migrations applied *first*

Ordering, learned the hard way. Migrations run *before* the seed, because the seed inserts into tables the migrations create. The container's init script does the opposite and guards on schema existence, so restarting a container does not re-seed and reports success either way.

PHASE 3 Local verification

E7 · GATE BEFORE ANY APPLICATION CHANGE

EXIT CHECKLIST

- ☐ Full suite green against a database that is not being rebuilt underneath it — a concurrent rebuild produces failures indistinguishable from real ones
- ☐ No concurrent test runs against the same database
- ☐ Row counts verified by query, never inferred from an exit code
- ☐ Every previously failing test either fixed or explained in writing

PHASE 4 Application layer

E7 BUILD · 2D SCORING

Work

- Repositories for CI, relationship and task-CI access, including a recursive traversal with cycle detection and an explicit recursion ceiling
- Structured route for grounded question answering, so the reader can query the estate rather than retrieve sentences about it
- Retrieval documents: prose only, CI path in the chunk prefix
- Resiliency and change-risk scoring derived from the graph — 2D

> EXIT CHECKLIST
>
> ☐ Traversal terminates on the planted cycle, proven by a dedicated test
>
> ☐ Recursion ceiling explicit; hitting it reports "at least N", never a bare N that reads as complete
>
> ☐ Grounded QA answers a structured question without retrieval, and says what is missing rather than denying a record that exists
>
> ☐ CMDB documents removed from the index only *after* the structured route works
>
> ☐ Resiliency reflects distinct physical parents, not node count

PHASE 5 Guardrails

E7

Prompt injection

Retrieved documents, work notes, ticket text and tool output are **data, never instructions**. The corpus is user-writable in any real deployment, so this is a live attack surface, not a theoretical one.

- System prompt states the boundary explicitly and instructs the model to describe injected instructions factually rather than obey them
- Evidence delivered in a labelled envelope, never concatenated into the instruction body
- Structured output enforced by server-side schema, so a model cannot smuggle instructions into a free-text field that something downstream executes
- No tool call, query or side effect may originate from retrieved content
- Number-drift guard rejects any figure not present in the evidence — an injected number cannot reach the reader
- Out-of-scope gate refuses questions with no estate signal before any model call
- An injection corpus of adversarial work notes lives in the test suite and every one is verified inert

Input and output validation

- Every request body validated by schema; unknown fields rejected, not ignored
- Maximum sizes on free-text input; truncation recorded as telemetry rather than silent
- Every model response validated against its schema before use; a malformed response is an error, not a warning
- Identifiers validated against the estate before they reach a query
- Parameterised SQL only — no string interpolation into a query, ever

Operational limits

- Per-provider spend ceilings in currency, soft warning before hard stop
- Request timeouts at every hop, shorter at the caller than the callee

- Rate limits per employee and per endpoint
- Result-set ceilings on every repository method, with the limit forwarded rather than dropped

> EXIT CHECKLIST
>
> ☐ Adversarial corpus in the suite; each case proven inert
>
> ☐ Number-drift guard enforced on every prose surface, not only the report
>
> ☐ No query built by string concatenation anywhere in the codebase
>
> ☐ Every timeout, size cap and rate limit configured, not defaulted
>
> ☐ Secrets absent from code, logs, transcripts and error messages — verified by grep, including shell traces

PHASE 6 RBAC and access control

E7

ROLE	MAY
Viewer	Read the estate, read investigations and their evidence. No model calls that cost money, no decisions
Engineer	Run investigations, ask questions, request capacity. Cannot approve a placement
Approver	Approve, reject or send back a recommendation. Their identity is recorded on the decision
Administrator	Model selection per role, budgets, indexing jobs, user administration

> EXIT CHECKLIST
>
> ☐ Every endpoint declares its required role; default is deny, never allow
>
> ☐ Authorisation enforced server-side; the interface hiding a control is presentation, not security
>
> ☐ A reviewer cannot submit a decision as another employee — identity comes from the token, not the payload
>
> ☐ Administrative endpoints separately gated and separately audited
>
> ☐ Database logins remain least-privilege; the application never holds blanket write permission
>
> ☐ Every privileged action writes an audit row with actor, time and outcome
>
> ☐ Tokens expire; signing key is environment-supplied and is not a development default

PHASE 7 Telemetry, cost and observability

E7

EXIT CHECKLIST

- ☐ Price table per provider and model with effective dates
- ☐ Cost computed and *persisted on the call row* at the price in force at that moment
- ☐ Latency histograms per model — p50, p95, p99 — not counters and not averages
- ☐ Budgets denominated in currency and enforced per provider as well as per workload
- ☐ Budget denial recorded as telemetry, not only raised as an exception
- ☐ Cost per investigation attributable and displayable
- ☐ Observability profile enabled in production and confirmed scraping
- ☐ Every dashboard panel has an alert rule and a named owner
- ☐ Structured logs with correlation identifiers across service boundaries

PHASE 8 Evaluation

2D RETRIEVAL · E7 HARNESS

EXIT CHECKLIST

- ☐ Golden set covers every investigation type, including refusals and adversarial cases
- ☐ Every case demonstrably able to fail — a case that cannot fail measures nothing
- ☐ Retrieval reported as recall@k, MRR and NDCG, with dense, sparse and hybrid separated
- ☐ The CMDB-document question answered with numbers before those documents are deleted
- ☐ Deterministic graders gate the result; the judge holds no numeric authority
- ☐ Multi-model comparison runs identical evidence across providers and reports quality, latency and cost as one artefact
- ☐ Evaluation runs in continuous integration; a regression fails the build
- ☐ Scheduled drift run against a fixed set, alerting on movement

PHASE 9 Host and deploy

E7

Order of operations — non-negotiable

1. Carry credentials out of the schema being dropped
2. Reset, then schema, then **migrations**, then seed
3. Restore credentials and verify the count is non-zero
4. Re-apply every grant — a reset drops all of them
5. Recreate the vector collection; stale vectors point at a corpus that no longer exists
6. Rebuild and restart only the services whose code changed
7. Smoke test before declaring success
8. Start indexing

EXIT CHECKLIST

- ☐ Row counts verified by query after every destructive step
- ☐ Credential count confirmed before the drop and after the restore
- ☐ Grants confirmed present by querying the permission catalogue, not by the script exiting zero
- ☐ Health endpoint green on every service
- ☐ One real investigation run end to end, with evidence, before the deploy is called done
- ☐ Rollback rehearsed, not assumed

PHASE 10 Run — failure and fallback

E7

FAILURE	REQUIRED BEHAVIOUR
Model provider down or rate limited	Ordered fallback chain; the fallback is recorded as telemetry; the answer states which model produced it
Embedding provider unavailable	Startup probe, not a per-call gamble; the index records which embedder actually built it and refuses to be queried by an incompatible one
Vector store unreachable	Degrade to structured answers and say so; never present a partial retrieval as complete
Database unavailable	Fail closed with a clear error; no cached number is ever presented as current
Budget exhausted	Refuse the model call, explain the ceiling, continue serving everything that does not need a model
Indexing interrupted	Resume from the last committed checkpoint; a run is leased and heartbeated so a dead worker is reclaimed rather than duplicated
Partial deploy	Detectable and stated — the system reports which corpus and which index it is serving

EXIT CHECKLIST

- ☐ Every failure mode above has a test that exercises it deliberately
- ☐ No silent degradation anywhere — degraded output says it is degraded
- ☐ Background jobs are resumable and idempotent; a repeated run does not duplicate work
- ☐ Job status vocabulary can express a deliberate stop, not only success and failure
- ☐ Runbook written for each: symptom, cause, action

STANDING Code practices

- **Comments explain why, never what.** A comment that restates the line is noise; a comment recording why an obvious approach was rejected is the most valuable line in the file.
- **No magic numbers.** Every threshold is a named constant with the reasoning attached.
- **Tests assert behaviour, not implementation**, and never a literal the generator produced. Fixtures are the contract.

- **A test that cannot fail is a defect.** Verify it fails when the thing it protects is broken. This has now happened six separate ways in one night, and the mechanisms share nothing but the outcome: a golden case whose answer had no competitors; `pytest.raises(Exception)` catching an `AttributeError` from calling the function wrongly; a suite whose fixtures all skipped, because a suite that skips is green; `pytest.skip` raising from `BaseException`, so an `except Exception` swallowed the skip; a resiliency score that fired correctly on every positive fixture and was still wrong, caught only by the one case that had to *stay quiet*; and an independent verification query that reproduced the same join shape as the code it was checking, so it passed while the bug was live. Treat "green" as unproven until the test has been seen to fail — and when writing the independent check, **verify differently, not merely separately**.
- **Never run tests during a database load.** A reset restarts every IDENTITY, so a test holding a row id loses its parent mid-run and fails on a foreign key — pointing at code that is fine. A false green costs a missed signal; a false red with a confident wrong cause costs an hour and, in CI, earns a `@flaky` decorator that buries it permanently.
- **Determinism by default** — seeded generation, no wall-clock or randomness in anything reproducible.
- **Migrations are idempotent and named.**
- **Grants live in the provisioning script**, never in a console session.
- **Secrets only in ignored environment files.** Never in code, chat, logs, or a shell trace.
- **Errors are typed and actionable** — what failed, why, what to do.
- **Stage by filename.** Three agents share one tree.
- **Verify by query, not by exit code.** The seed that "succeeded" and loaded nothing is the reason this line exists.
